CAMPUS TRADE

Especificación técnica

Arquitectura, componentes y APIs para el prototipo web de compra, venta, renta, intercambio y donación universitaria.

Proyecto CampusTrade	Equipo Equipo 3 MEX-PER
Área responsable Ingeniería en Sistemas Computacionales (ISC)	Versión 1.0 · Fase alfa

Propósito del documento
Define la solución técnica del equipo ISC. Distingue las capacidades implementadas en el prototipo alfa de los contratos y componentes propuestos para una implementación con servidor y base de datos.

1. Alcance técnico y criterios

CampusTrade es una aplicación web orientada a la comunidad universitaria para publicar y localizar artículos o servicios bajo cuatro modalidades: venta, renta, intercambio y donación. El diseño se deriva de los requerimientos funcionales validados y del prototipo React entregado por ISC.

1.1 Estado de la solución

Elemento	Estado en fase alfa	Decisión técnica
Interfaz web	Implementado	React 18 renderiza el catálogo, detalle, filtros, favoritos, perfil, mensajería y formulario de publicación.
Datos	Simulados en memoria	El repositorio listingRepository encapsula el acceso. Puede sustituirse por peticiones HTTP sin reescribir vistas.
Autenticación	No conectada	Se especifica API y token para la siguiente fase; no se deben presentar datos reales ni contraseñas en el prototipo.
Base de datos	No incluida	Se propone un modelo relacional y almacenamiento de imágenes separado para una fase posterior.
Despliegue	Local / desarrollo	Vite sirve el proyecto; el compilado puede publicarse en un hosting estático cuando exista API disponible.

1.2 Principios de arquitectura

• Separación de responsabilidades: la interfaz no conoce la fuente concreta de los datos; consume un repositorio/servicio.

• API REST versionada bajo /api/v1, con recursos nombrados en plural y respuestas JSON.

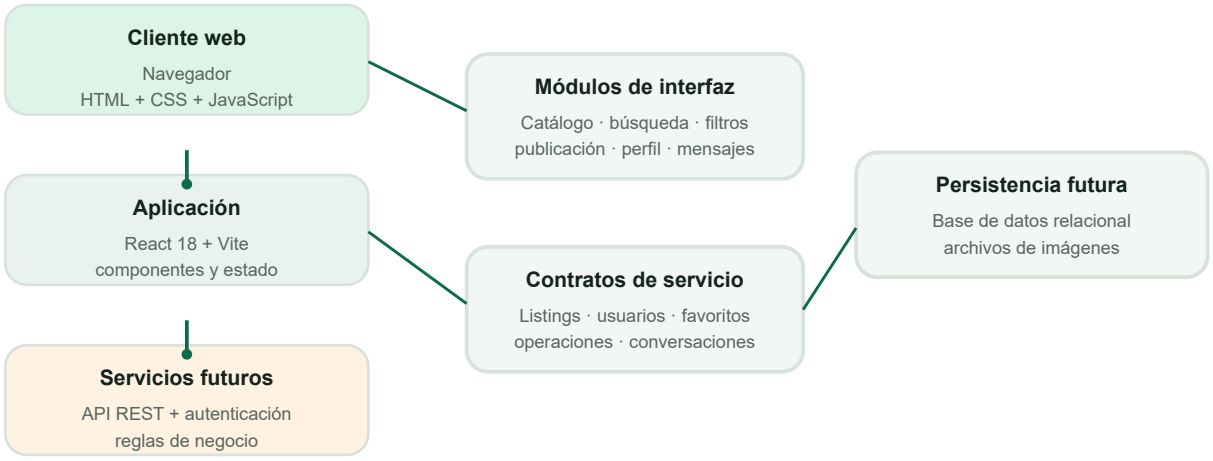
• Diseño mobile-first y componentes reutilizables para que nuevas pantallas o modalidades no requieran duplicar lógica.

• Validación en cliente para experiencia inmediata y validación obligatoria en servidor para conservar integridad y seguridad.

> Límite de alcance
> Esta especificación cubre exclusivamente la parte técnica de ISC. No incorpora entregables de administración, análisis comercial ni actividades de producción ajenas al sistema.

2. Arquitectura del sistema

La solución se organiza en tres capas lógicas. Actualmente existen las dos primeras en forma de interfaz y repositorio simulado; las capas de API y persistencia son la ruta de evolución para conectar el producto a usuarios institucionales.



Separación propuesta: presentación → servicios → datos. El prototipo actual utiliza un repositorio simulado.

Figura 1. Arquitectura objetivo y puntos de sustitución del prototipo.

2.1 Capas y responsabilidades

Capa	Responsabilidad	Tecnologías / contratos
Presentación	Mostrar catálogo, capturar formularios, navegación, filtros y retroalimentación de acciones.	HTML5, CSS3, React 18, JavaScript ES2022.
Aplicación	Gestionar estado local, transformar entradas de usuario y delegar operaciones al repositorio o API.	Componentes React, hooks, módulos ES, Vite.
Servicios	Aplicar autenticación, autorización, reglas de operación, validación y auditoría.	API REST JSON; implementación propuesta: Node.js + Express o framework equivalente.
Datos	Persistir usuarios, publicaciones, mensajes, operaciones, calificaciones y reportes.	PostgreSQL o MySQL; almacenamiento de objetos para imágenes.

La interfaz no debe conectarse directamente a la base de datos. Únicamente el servicio de aplicación tendrá credenciales de persistencia y aplicará las reglas de acceso.

3. Componentes técnicos principales

3.1 Frontend implementado

Módulo	Responsabilidad concreta	Entrada / salida
main.jsx	Punto de arranque; monta la aplicación React en el nodo raíz del documento HTML.	Carga App en #root.
App.jsx	Coordina pantallas, navegación del prototipo, búsqueda, filtros, favoritos, detalle y flujos simulados.	Estado de vista, consulta, filtros y colección de favoritos.
mockListings.js	Contiene publicaciones de demostración, categorías y opciones de filtro.	Datos locales usados exclusivamente en alfa.
listingRepository.js	Define operaciones asíncronas de listado, consulta, creación y actualización sin acoplarse a la fuente de datos.	Promesas con entidades de publicación.
styles.css	Sistema visual responsive, tarjetas, formularios, navegación y accesibilidad visual.	Estilos globales y por componente.

3.2 Dependencias y compilación

Tecnología	Versión / uso	Justificación
React	18.x	Biblioteca de interfaz basada en componentes y estado declarativo.
React DOM	18.x	Renderizado de la aplicación React en navegador.
Vite	5.x	Servidor de desarrollo, recarga rápida y empaquetado de la aplicación.
HTML5 / CSS3	Estándares web	Estructura compatible con navegadores modernos y diseño responsive.
JavaScript	ES2022	Lógica de interfaz, módulos y llamadas asíncronas futuras.

3.3 Estructura esperada al incorporar servidor

```
frontend/                backend/
  src/                     src/
    components/              routes/        # rutas HTTP
    pages/                   controllers/   # entrada/salida JSON
    services/                services/      # reglas de negocio
    repositories/            repositories/  # consultas a datos
    data/                    middleware/    # auth, validación, errores
                             models/        # entidades y migraciones
```

4. Especificación de la API REST

Los endpoints siguientes están propuestos para la integración posterior. El prototipo alfa usa el repositorio local, por lo que ninguna llamada requiere aún una conexión de red. La API recibirá y devolverá JSON UTF-8; todas las rutas se agrupan bajo /api/v1.

4.1 Convenciones de contrato

Regla	Especificación
Autorización	Authorization: Bearer <JWT> en rutas privadas. El JWT identifica usuario, rol y expiración.
Fechas	ISO 8601 en UTC, por ejemplo 2026-10-14T18:30:00Z.
Paginación	Parámetros page y limit; respuesta con items, page, limit y total.
Errores	Objeto { error: { code, message, fields? } }; no revelar trazas ni datos internos.

4.2 Recursos y operaciones

Método y ruta	Autorización	Uso
POST /auth/register	Pública	Registra una cuenta con correo institucional y datos académicos.
POST /auth/login	Pública	Valida credenciales y devuelve token de acceso y perfil básico.
GET /listings	Pública	Lista publicaciones con query, categoría, carrera, modalidad, condición, rango de precio, estado y orden.
POST /listings	Usuario	Crea una publicación; valida campos obligatorios y asigna propietario.
GET /listings/{id}	Pública	Devuelve detalles, imágenes, estado y resumen público del vendedor.
PATCH /listings/{id}	Propietario	Actualiza campos permitidos o disponibilidad de la publicación.
POST /listings/{id}/favorites	Usuario	Marca o desmarca publicación favorita para el usuario autenticado.
POST /conversations	Usuario	Crea o reutiliza una conversación entre interesado y vendedor.
POST /operations	Usuario	Solicita renta, intercambio, donación o reserva de una publicación.
POST /reports	Usuario	Registra reporte de fraude, spam, contenido prohibido o incumplimiento.

5. Detalle de datos y ejemplos de API

5.1 Consulta de publicaciones

```
GET /api/v1/listings?q=calculo&career=ISC&modality=venta&condition=bueno&page=1&limit=12

200 OK
{
  "items": [{"id": "lst_102", "title": "Libro de Cálculo",
             "price": 180.00, "currency": "MXN", "modality": "sale",
             "condition": "good", "status": "available", "imageUrl": "..."}],
  "page": 1, "limit": 12, "total": 1
}
```

5.2 Creación de publicación

Campo	Tipo / regla	Ejemplo
title	string, 8–100 caracteres	Calculadora científica Casio
description	string, 20–1,500 caracteres	Funciona correctamente; incluye funda.
categoryId	UUID / id existente	cat_electronica
career	enum o catálogo	ISC
condition	new | like_new | good | fair	good
modality	sale | rental | exchange | donation	sale
price	decimal ≥ 0; obligatorio para venta/renta	350.00
imageIds	arreglo de referencias ya cargadas	["img_a1", "img_a2"]

La carga de archivos debe usar POST /uploads con multipart/form-data. El servicio valida tipo, tamaño y contenido antes de devolver una referencia segura; la creación de la publicación acepta dichas referencias y no una ruta de disco enviada por el cliente.

5.3 Códigos de respuesta

Código	Cuándo se devuelve	Acción del cliente
200 / 201	Lectura o creación exitosa.	Actualizar interfaz con la entidad recibida.
400	Formato, parámetros o regla de negocio inválidos.	Mostrar mensaje por campo cuando aplique.
401 / 403	Sesión ausente/expirada o permiso insuficiente.	Solicitar inicio de sesión o ocultar acción.
404	Recurso inexistente o no visible.	Mostrar estado vacío y volver al catálogo.
409	Conflicto: artículo ya reservado o estado incompatible.	Refrescar detalle y comunicar cambio.
429 / 500	Límite de solicitudes o fallo controlado.	Reintentar de forma moderada y no perder el formulario.

6. Modelo de datos propuesto

La persistencia se plantea con una base relacional porque el sistema requiere consistencia entre publicaciones, operaciones, mensajes y calificaciones. Las imágenes deben guardarse fuera de la base de datos; la tabla conserva solamente metadatos y URL/clave de objeto.

6.1 Entidades principales

Entidad	Campos técnicos clave	Relaciones
User	id, institutional_email, matricula_hash, career_id, password_hash, role, verified_at, created_at	1:N Listing, Favorite, Message, Operation.
Listing	id, seller_id, category_id, title, description, modality, condition, price, status, published_at	N:1 User; 1:N ListingImage, Operation, Report.
Category	id, name, parent_id, active	1:N Listing; permite categorías y subcategorías.
ListingImage	id, listing_id, object_key, alt_text, position, mime_type	N:1 Listing; no expone claves internas directamente.
Favorite	user_id, listing_id, created_at	Clave única compuesta user_id + listing_id.
Conversation / Message	id, listing_id, participant ids; message.id, sender_id, body, sent_at, read_at	Conversación asociada opcionalmente a una publicación.
Operation	id, listing_id, requester_id, type, status, rental_start, rental_end, offered_listing_id	Mantiene la transición y la trazabilidad de la transacción.
Rating / Report	rating: score, comment; report: reason, detail, status	Se habilitan tras operación o sobre usuario/publicación.

6.2 Estados controlados

Objeto	Estados permitidos	Regla
Listing.status	draft, available, reserved, sold, rented, exchanged, donated, cancelled	Solo propietario o proceso autorizado puede modificarlo; los estados terminales no reciben nuevas solicitudes.
Operation.status	pending, accepted, rejected, completed, cancelled, expired	La aceptación valida que el listing permanezca disponible; la finalización habilita calificación.
Report.status	open, reviewing, resolved, dismissed	Solo perfil con rol de moderación administra el cierre.

Integridad mínima
Usar claves foráneas, índices en Listing(status, modality, category_id, published_at), correos institucionales únicos y una transacción al aceptar una operación para evitar doble reserva.

7. Seguridad, calidad y configuración

7.1 Controles de seguridad requeridos

Riesgo	Control técnico obligatorio
Acceso no autorizado	Contraseñas con Argon2id o bcrypt; tokens JWT de vida corta; renovación controlada; cierre de sesión en cliente.
Suplantación	Verificación de dominio institucional y estado verified_at antes de publicar, iniciar chat o solicitar operaciones.
Manipulación de recursos	Autorización por propietario/rol en cada endpoint; nunca confiar en userId enviado por el navegador.
Entradas maliciosas	Esquemas de validación en servidor, sanitización de texto mostrado y consultas parametrizadas; protección contra XSS/SQLi.
Carga de imágenes	Lista permitida de MIME, límite de tamaño, renombrado interno, análisis antivirus si se habilita infraestructura y URLs firmadas para acceso.
Abuso	Rate limit en login, registro, mensajería y reportes; registro de eventos de seguridad sin guardar secretos.

7.2 Variables de configuración

```
# .env (no versionar ni exponer en el frontend)
API_PORT=3000
DATABASE_URL=postgresql://usuario:secreto@host:5432/campustrade
JWT_SECRET=<secreto_largo>
JWT_EXPIRES_IN=15m
CORS_ORIGIN=https://campustrade.ejemplo.mx
OBJECT_STORAGE_BUCKET=campustrade-images
MAX_UPLOAD_MB=5
```

En frontend, una URL pública de API se declara con el prefijo que requiera el empaquetador (por ejemplo, VITE_API_BASE_URL). Secretos, credenciales de base de datos y claves de firma permanecen exclusivamente en el servidor.

7.3 Requisitos de calidad técnica

Aspecto	Criterio verificable
Compatibilidad	Últimas dos versiones de Chrome, Edge, Firefox y Safari; diseño usable desde 360 px de ancho.
Rendimiento	Catálogo paginado; imágenes optimizadas; objetivo de carga inicial menor a 3 s en red móvil promedio.
Accesibilidad	Etiquetas de formulario, contraste suficiente, foco visible y navegación por teclado en acciones principales.
Observabilidad	Errores controlados con identificador de solicitud; métricas de latencia y registros de cambios de estado.

8. Implementación, pruebas y evolución

8.1 Ejecución del prototipo

Paso	Comando / resultado esperado
Instalar dependencias	npm install · obtiene las dependencias declaradas en package.json.
Iniciar desarrollo	npm run dev · Vite informa una URL local, normalmente http://localhost:5173/.
Abrir aplicación	Usar la URL de Vite indicada en la terminal; no abrir index.html directamente con file://, porque los módulos requieren el servidor de desarrollo.
Compilar	npm run build · genera los archivos estáticos para revisión o publicación.

8.2 Casos técnicos de prueba para la fase alfa

ID	Caso	Resultado esperado
PT-01	Carga inicial del catálogo	Se renderizan publicaciones simuladas, categorías y navegación sin errores visibles.
PT-02	Búsqueda por palabra	La lista muestra sólo coincidencias de título/descripción o el estado vacío correspondiente.
PT-03	Filtro por modalidad y condición	Los criterios se aplican de forma acumulada y se pueden restablecer.
PT-04	Detalle y favoritos	El detalle abre la entidad seleccionada y el favorito modifica sólo el estado de sesión simulado.
PT-05	Publicación simulada	El formulario valida requeridos y agrega el elemento al repositorio local, sin persistir tras recarga.
PT-06	Diseño responsive	Las acciones principales son accesibles en vista móvil y escritorio.

8.3 Hoja de ruta de integración

Prioridad	Siguiente incremento técnico	Criterio de terminación
Alta	Reemplazar repository simulado por cliente HTTP y exponer GET /listings, GET /listings/{id}, POST /listings.	Catálogo y publicación consultan una API de prueba con datos persistentes.
Alta	Registro, login y verificación institucional.	Rutas privadas protegidas; perfil autenticado; contraseñas no viajan ni se almacenan en texto plano.
Media	Mensajería, favoritos y operaciones con reglas de estado.	Cada acción persiste y respeta propietario, disponibilidad y permisos.
Media	Moderación, calificaciones, métricas y despliegue continuo.	Reportes administrables, bitácora de cambios y ambiente de producción configurado.

Cierre técnico
La arquitectura permite demostrar el flujo de CampusTrade hoy y evolucionar a una aplicación con API y base de datos sin sustituir el diseño de la interfaz. Cada componente futuro se incorpora detrás de contratos explícitos y verificables.